

numberAI: Building an Image Classifier from Scratch

Project Documentation

May 9, 2026

Abstract

This document outlines the development, architecture, and findings of the **numberAI** project. The objective of this project is to recreate machine learning functionality from scratch, without relying on pre-trained models or established neural network structures. By utilizing simple matrix multiplication and a trial-and-error training approach, the project aims to categorize number images. Additionally, it details the planned second phase involving quantization and distillation for deployment on resource-constrained embedded systems such as Arduino.

Contents

1	Introduction	2
1.1	Project Motivation	2
1.2	Goals and Phases	2
2	Phase 1: Architecture Iterations	2
2.1	Version 0: Initial Attempt	2
2.1.1	Architecture and Methodology	2
2.1.2	Results and Failures	2
2.2	Version 1.0: Non-Linearity and Optimization	3
2.2.1	Architecture Upgrades	3
2.2.2	Methodology Upgrades	3
2.2.3	Version 1.0 Training Results	3
2.3	Version 1.1: Vectorized Blame Assignment (Backpropagation)	5
2.3.1	Methodology Upgrades	5
2.3.2	Version 1.1 Training Results	5
2.4	Version 2.0 (Planned)	5
3	Phase 2: Embedded Systems Optimization	5
3.1	Quantization	6
3.2	Distillation	6
4	Discoveries and Challenges	6
4.1	Matrix Size Optimization	6
4.2	Trial and Error Efficiency	6
5	Conclusion	6

1 Introduction

1.1 Project Motivation

Modern machine learning often relies on complex, pre-defined architectures and pre-trained weights. The motivation behind numberAI is to deconstruct this paradigm and build an image classifier from the absolute basics to fully understand the underlying mechanics of neural networks.

1.2 Goals and Phases

The project is divided into two main phases:

- **Phase 1: Foundational Discovery.** To categorize number images using pure matrix multiplication. This phase avoids pre-defined structures, instead relying on trial and error to iteratively discover the optimal matrix size and network topology.
- **Phase 2: Embedded Systems Deployment.** To run the developed classification model on embedded systems with limited resources (e.g., Arduino). This requires applying model compression techniques such as *quantization* and *distillation* to reduce memory footprint and computational load.

2 Phase 1: Architecture Iterations

As the project employs a trial-and-error discovery method, the model architecture and training methodology evolved chronologically through multiple versions. This section documents the progression of the model, detailing the problems encountered in each version and the subsequent upgrades.

2.1 Version 0: Initial Attempt

The very first attempt (**Version 0**) served as a baseline to test the pure matrix multiplication concept without any additional complexity.

2.1.1 Architecture and Methodology

- **Linear Topology:** The network consisted of two weight matrices ($H1$ and $H2$) with no activation functions between them.
- **I/O Inefficiency:** The training loop read and wrote the matrices to the disk at every single iteration.

2.1.2 Results and Failures

Version 0 was ultimately unsuccessful due to several critical flaws:

- **Linearity Collapse:** Without a non-linear activation function, the two-layer network was mathematically pointless. Sequential linear matrix multiplications can be collapsed into a single matrix, meaning no deep representations were being learned.

- **Performance Bottleneck:** Reading and writing matrices to the disk at every step made the training process extremely slow.
- **Gradient Explosion:** The training never finished because the loss would eventually hit NaN due to gradient explosions. Consequently, no usable model was produced for testing.

2.2 Version 1.0: Non-Linearity and Optimization

To address the fatal flaws of Version 0, several major upgrades were introduced in **Version 1.0**.

2.2.1 Architecture Upgrades

The structure for Version 1.0 introduced non-linearity:

- **Input Layer:** Flattened representation of the input image.
- **Hidden Layers:** Two primary weight matrices ($H1$ and $H2$). Dimensions were iteratively tested (e.g., $256 \times 32 \rightarrow 32 \times 10$, and later $4096 \times 128 \rightarrow 128 \times 10$).
- **Activation Function:** ReLU (Rectified Linear Unit) was introduced between the hidden layers to prevent the linearity collapse seen in Version 0.
- **Output Layer:** Vector of size 10×1 , representing the classification scores for digits 0-9.

2.2.2 Methodology Upgrades

- **Optimized I/O:** The matrix reading and writing methods were optimized to keep weights in memory during the training loop, massively speeding up training.
- **Numerical Gradient Training:** The loss function calculates the sum of Euclidean distances between the actual output vector ($\mathbf{A}$) and expected output ($\mathbf{EA}$):

$$\text{Loss} = \sum_{k=0}^9 \sqrt{\sum_{n=0}^9 (A_{k,n} - EA_{k,n})^2} \quad (1)$$

The weight was then updated using a numerical gradient approximation (trial-and-error):

$$H_{m,n} := H_{m,n} - \left(\frac{\text{Loss}_{o+\text{change}} - \text{Loss}_o}{\text{change}} \right) \cdot \alpha \quad (2)$$

2.2.3 Version 1.0 Training Results

After training the Version 1.0 architecture, a measurable drop in Shannon entropy for the weight matrices was observed:

- **Pre-training Entropy:** $H1 \approx 6.63$, $H2 \approx 6.45$
- **Post-training Entropy:** $H1 \approx 3.05$, $H2 \approx 3.23$

This lower entropy indicates that the trial-and-error process successfully converged on structured patterns within the dataset. A visualization of the training loss can be seen below:

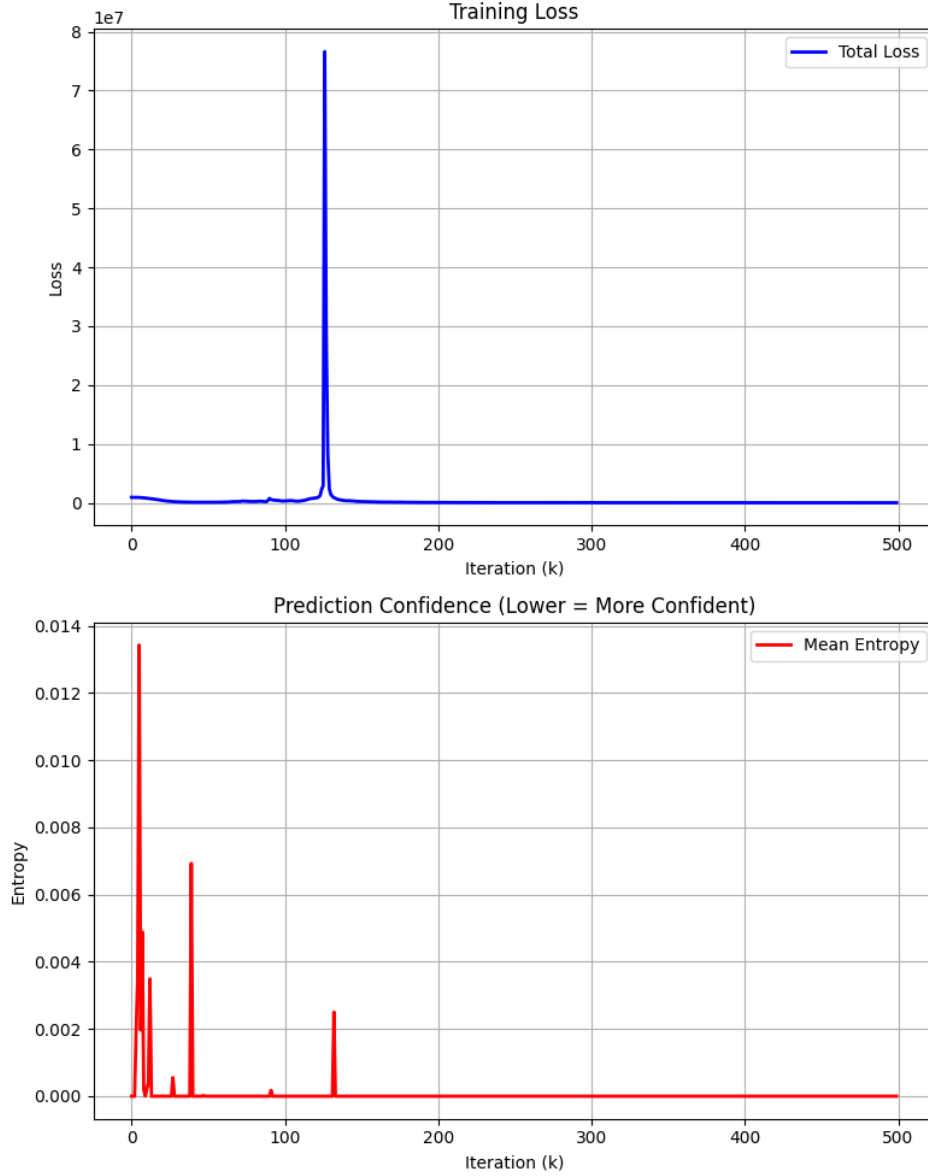


Figure 1: Version 1.0 Training Loss Progression

Overfitting Discovery: While the training process converged, evaluating the model revealed a severe case of overfitting. The model successfully categorized all images in the training dataset, achieving an accuracy of 100.00% (100/100). However, when exposed to new, unseen number images from the verification dataset, it completely failed, yielding an accuracy of 0.00% (0/10). This indicates that the Version 1.0 architecture effectively memorized the specific pixels of the training data but failed to generalize the underlying visual features of the numbers.

2.3 Version 1.1: Vectorized Blame Assignment (Backpropagation)

To address the severe overfitting and slow training times observed in Version 1.0, the training methodology was significantly overhauled in **Version 1.1**. The numerical gradient approach (trial-and-error finite difference) was replaced with a heavily optimized, mathematically grounded blame assignment method.

2.3.1 Methodology Upgrades

The adjustment logic was optimized using vectorized blame assignment (backpropagation):

- **H2 Adjustment:** The error vector is multiplied by the transposed middle neuron vector (from $H1$) to compute the gradient for all parameters of $H2$ simultaneously.
- **H1 Adjustment:** The error vector is multiplied by the transposed $H2$ matrix, passed through the derivative of the ReLU function, and then multiplied by the transposed input vector (I). This allows the simultaneous adjustment of all $H1$ parameters.

This vectorized approach eliminates the need to perturb weights one-by-one, replacing millions of individual forward passes with a single efficient backward pass.

2.3.2 Version 1.1 Training Results

The implementation of vectorized blame assignment yielded two major breakthroughs:

1. **Blazing Fast Training:** The training time was drastically reduced to approximately 30.6 seconds (down from over 15 minutes in Version 1.0), completing 500 iterations over the entire dataset with ease.
2. **Generalization over Memorization:** The updated weight adjustment logic successfully prevented the model from merely memorizing pixels. Evaluation metrics showed:
 - **Training Dataset Accuracy:** 100.00% (100/100)
 - **Verification Dataset Accuracy:** 60.00% (6/10)

While 60% accuracy on the verification set indicates there is still room for improvement, it definitively proves that the Version 1.1 network is finally learning to generalize visual patterns rather than overfitting the training data.

2.4 Version 2.0 (Planned)

[Placeholder: Describe changes made in Version 2.0 to improve upon the shortcomings of Version 1.1.]

3 Phase 2: Embedded Systems Optimization

To deploy this model on an Arduino, the network must be significantly compressed.

3.1 Quantization

[Placeholder: Describe the process of converting floating-point matrix weights to lower precision formats (e.g., 8-bit integers) and the resulting impact on accuracy.]

3.2 Distillation

[Placeholder: Describe how a smaller, more efficient model (student) is trained to replicate the behavior of the larger, successful trial-and-error model (teacher).]

4 Discoveries and Challenges

4.1 Matrix Size Optimization

[Placeholder: Discuss which matrix dimensions yielded the best balance between accuracy and computational complexity.]

4.2 Trial and Error Efficiency

[Placeholder: Discuss the challenges faced when training without traditional gradient descent. What heuristics were discovered to speed up convergence?]

5 Conclusion

[Placeholder: Summarize the final outcomes of the project, whether the goals were fully met, and potential future improvements.]